

Продвинутые примитивы синхронизации: SemaphoreSlim, ReaderWriterLockSlim, ManualResetEventSlim

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?



Ставим в чат "+", если все хорошо
"-", если есть проблемы



Михаил Дмитриев

Ведущий программист НИПК Электрон

- 5+ лет опыта промышленной разработки на платформе Net
- 20+ лет опыта работы в IT сфере
- Разрабатываю и поддерживаю приложения для работы с радиологическими комплексами

➤ <https://t.me/sf321>

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе в
телеграмме



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Управление памятью
на уровне ядра

Асинхронность и
многопоточность
для экспертов

Внутренние
механизмы CLR

Профилирование и
экстремальная
оптимизация

Современные
парадигмы и
инструменты эксперта

Проектный
модуль

ФИНИШ



Маршрут вебинара

Многопоточность

Проблемы

Особенности Slim-версий
примитивов синхронизации

Асинхронный семафор

ReaderWriterLockSlim

ManualResetEventSlim

Цели вебинара

К концу занятия вы сможете

1. Понимать чем отличаются slim - версии примитивов синхронизации

2. Рассмотреть особенности SemaphoreSlim, ReaderWriterLockSlim, WaitHandleEventSlim

3. Максимально эффективно использовать ресурсы системы, избегая блокировок потоков

Смысл

Для чего вам это уметь

Делать выбор оптимального примитива синхронизации под вашу задачу

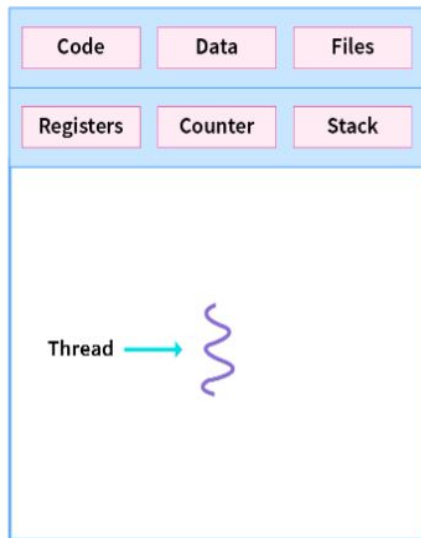




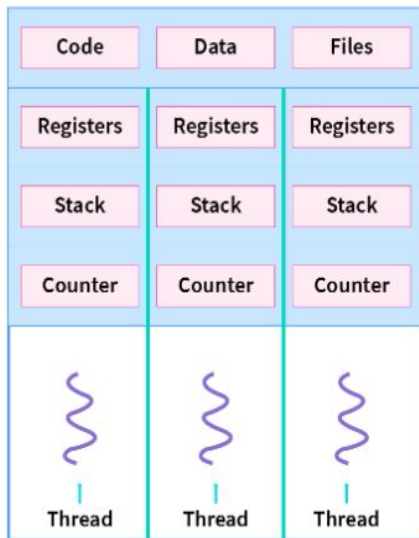
Тестирование

Введение

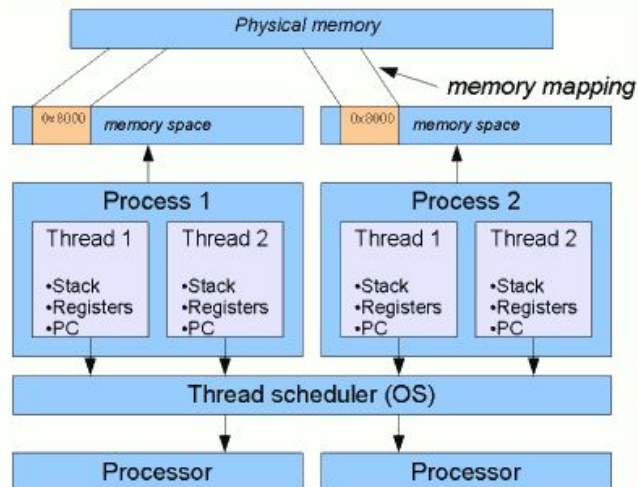
Процессы , потоки



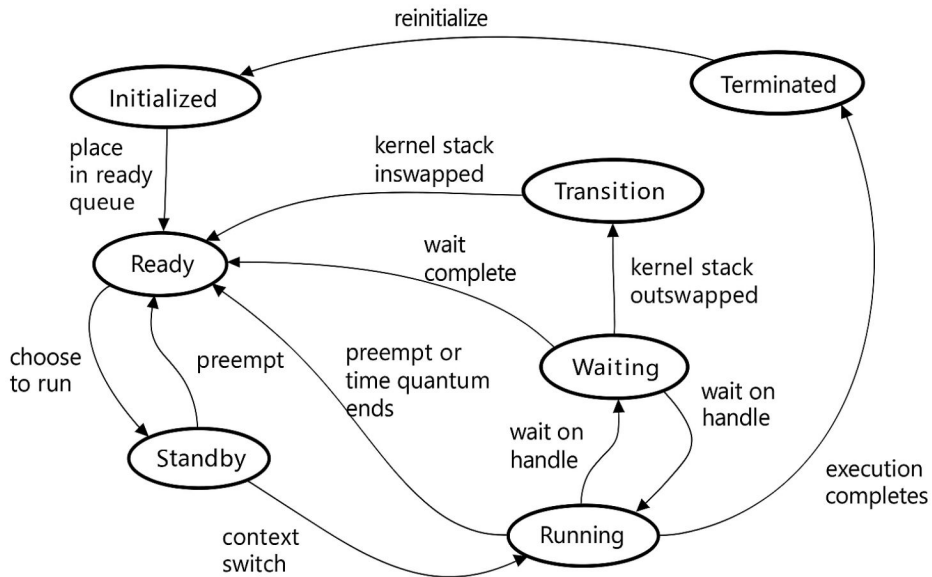
Single-threaded process



Multithreaded process

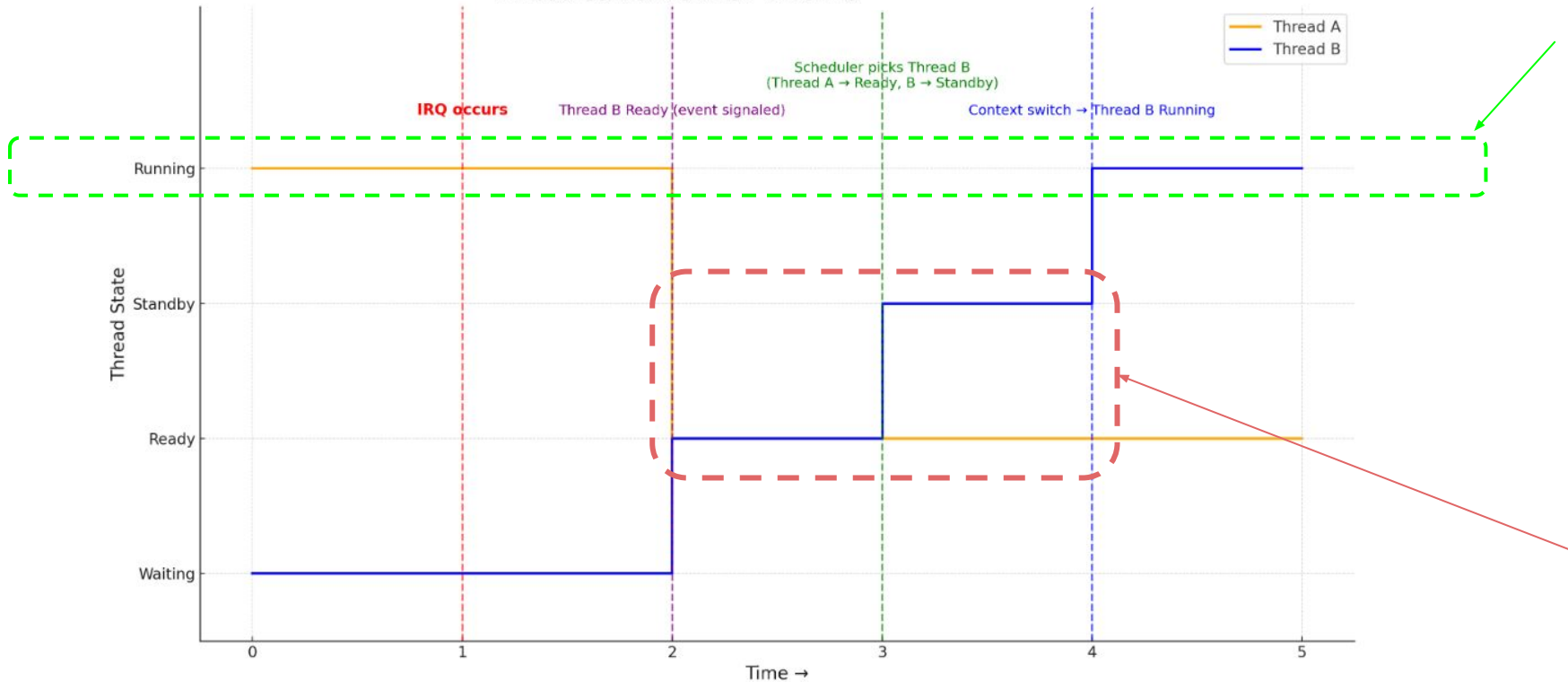


Состояния потока в рамках работы ОС



Context switch

Thread Context Switch Timeline



Проблема синхронизации

Начальное состояние:

$y = 0; x = 1;$

Параллельные операции

$\{y = x + 1;\} \parallel \{x = y + 2;\}$

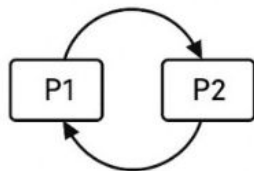
Возможные конечные состояния:

1. $y == 2; x == 2;$
2. $y == 3; x == 2;$
3. $y == 2; x == 4;$



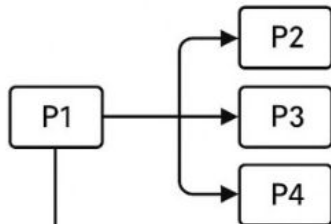
Deadlock, busy waiting and starvation

Deadlock



waiting for each
other's resource

Starvation

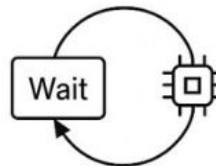


others repeatedly
get access

Waiting

P1 starves —
never gets access

Busy Waiting



CPU spinning
(busy waiting)

Deadlock

```
1 using System;
2 using System.Threading;
3
4 class Program
5 {
6     static Semaphore semaphore = new Semaphore(1, 1); // only 1 thread can enter
7
8     static void Main()
9     {
10         // Thread 1 acquires semaphore
11         semaphore.WaitOne();
12         Console.WriteLine("Main thread acquired semaphore");
13
14         // Start another thread that also tries to acquire it
15         new Thread(OtherThread).Start();
16
17         // The main thread tries to acquire it again → DEADLOCK
18         Console.WriteLine("Main thread trying to acquire semaphore again...");
19         semaphore.WaitOne(); // <-- will block forever
20         Console.WriteLine("This line will never be reached");
21     }
22
23     static void OtherThread()
24     {
25         Console.WriteLine("Other thread waiting...");
26         semaphore.WaitOne();
27         Console.WriteLine("Other thread acquired semaphore");
28         semaphore.Release();
29     }
30 }
```

Starvation

```
1 using System;
2 using System.Threading;
3
4 class Program
5 {
6     static Semaphore semaphore = new Semaphore(1, 1); // only one thread can enter
7     static int shared = 0;
8
9     static void Main()
10    {
11        // Start one "greedy" thread that holds semaphore for a long time
12        new Thread(GreedyThread) { IsBackground = true }.Start();
13
14        // Start many short "worker" threads that rarely get access
15        for (int i = 0; i < 5; i++)
16            new Thread(WorkerThread) { IsBackground = true }.Start();
17
18        Console.ReadLine();
19    }
20
21    static void GreedyThread()
22    {
23        while (true)
24        {
25            semaphore.WaitOne();
26            // simulate long work
27            Console.WriteLine("Greedy thread working...");
28            Thread.Sleep(2000);
29            semaphore.Release();
30            Thread.Sleep(10); // almost no pause -> keeps reacquiring fast
31        }
32    }
33
34    static void WorkerThread()
35    {
36        while (true)
37        {
38            if (semaphore.WaitOne(100))
39            {
40                shared++;
41                Console.WriteLine($"Worker {Thread.CurrentThread.ManagedThreadId} got access,");
42                Thread.Sleep(200); // short work
43                semaphore.Release();
44            }
45            else
46            {
47                Console.WriteLine($"Worker {Thread.CurrentThread.ManagedThreadId} starved...")
48            }
49            Thread.Sleep(100);
50        }
51    }
52 }
```

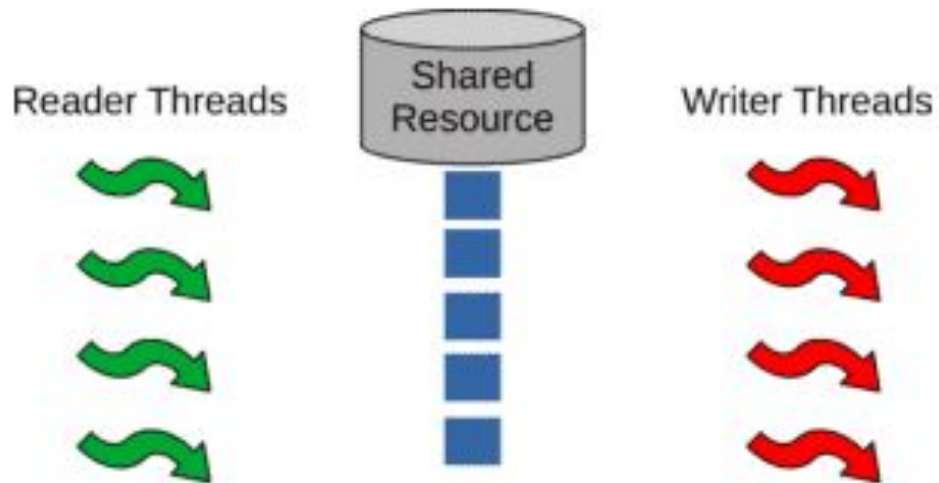


Busy wait

```
1 using System;
2 using System.Threading;
3
4 class Program
5 {
6     static Semaphore semaphore = new Semaphore(1, 1);
7
8     static void Main()
9     {
10         // Take the semaphore so no one else can enter
11         semaphore.WaitOne();
12         Console.WriteLine("Main thread acquired semaphore");
13
14         // Start another thread that will busy-wait
15         new Thread(BusyWaitThread).Start();
16
17         // Hold the semaphore for a while
18         Thread.Sleep(3000);
19         semaphore.Release();
20
21         Console.WriteLine("Main thread released semaphore");
22         Console.ReadLine();
23     }
24
25     static void BusyWaitThread()
26     {
27         Console.WriteLine("Busy thread starting busy-wait loop...");
28
29         // naive busy-wait loop
30         while (!semaphore.WaitOne(0))
31         {
32             // The thread keeps looping, burning CPU
33             Console.Write(".");
34             Thread.SpinWait(100000); // simulate tight spin
35         }
36
37         Console.WriteLine("\nBusy thread acquired semaphore!");
38         semaphore.Release();
39     }
40 }
```

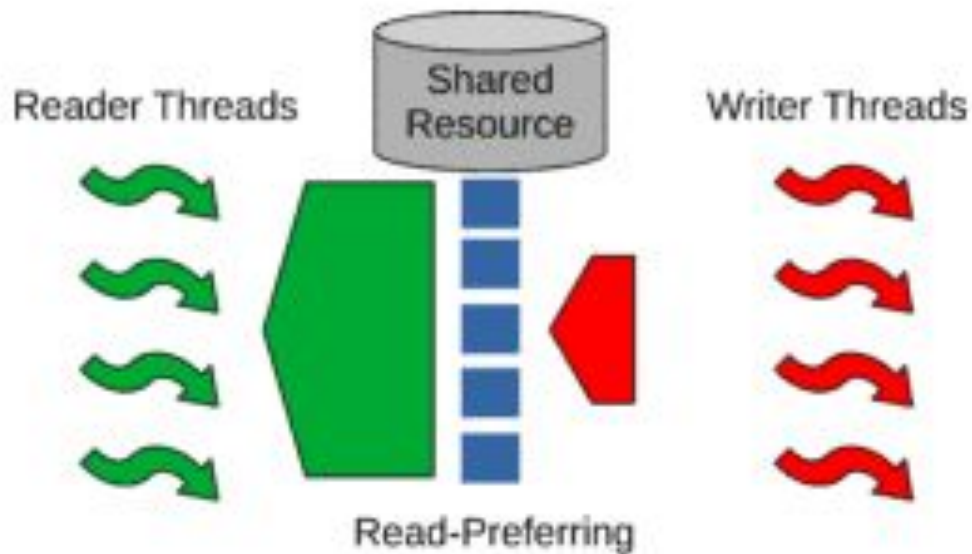


Проблема читателей-писателей



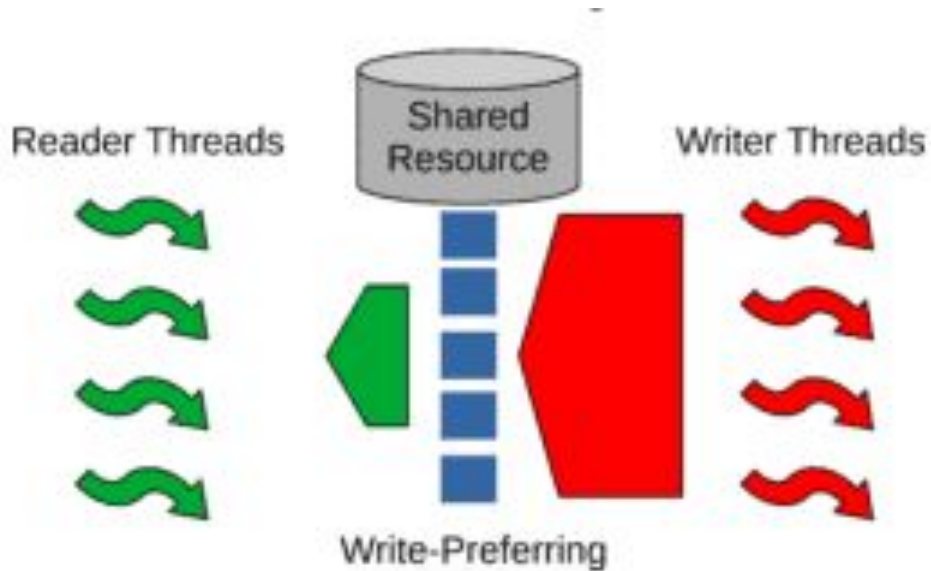
<https://www.baeldung.com/cs/readers-writers-problem>

Решение Reader Bias



<https://www.baeldung.com/cs/readers-writers-problem>

Решение Writer Bias



<https://www.baeldung.com/cs/readers-writers-problem>

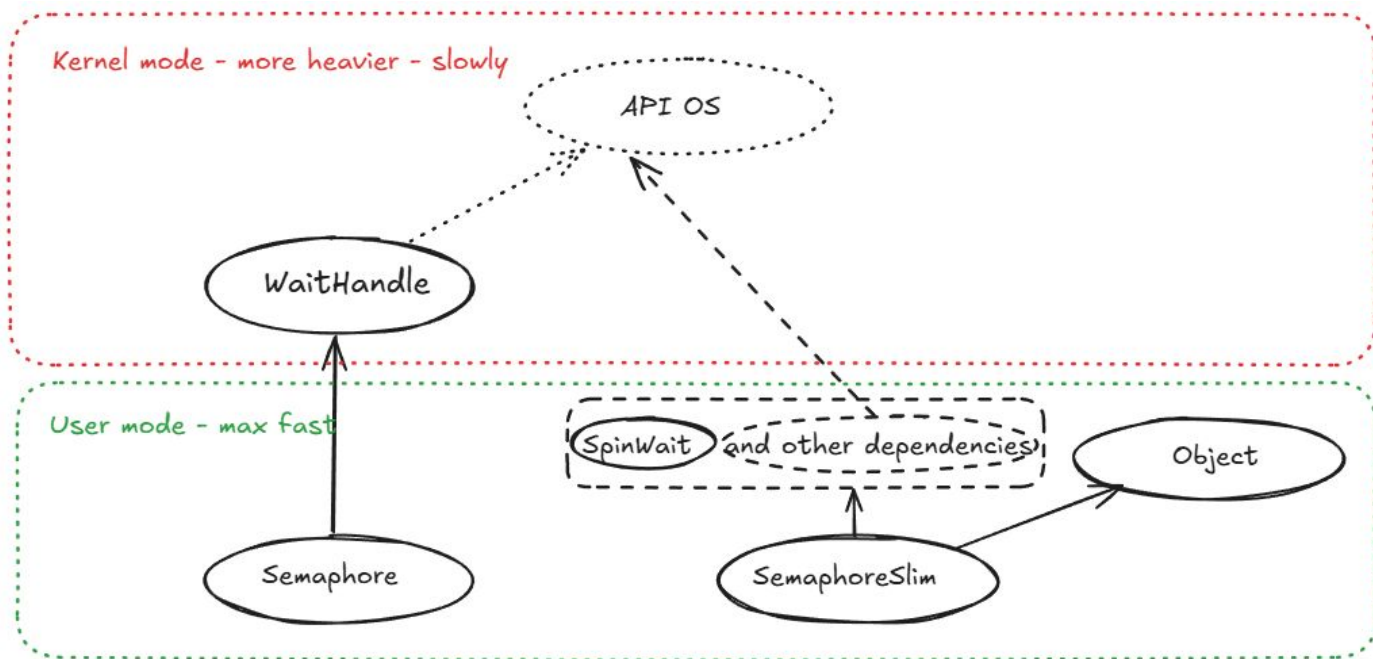
Примитивы синхронизации

Примитивы синхронизации

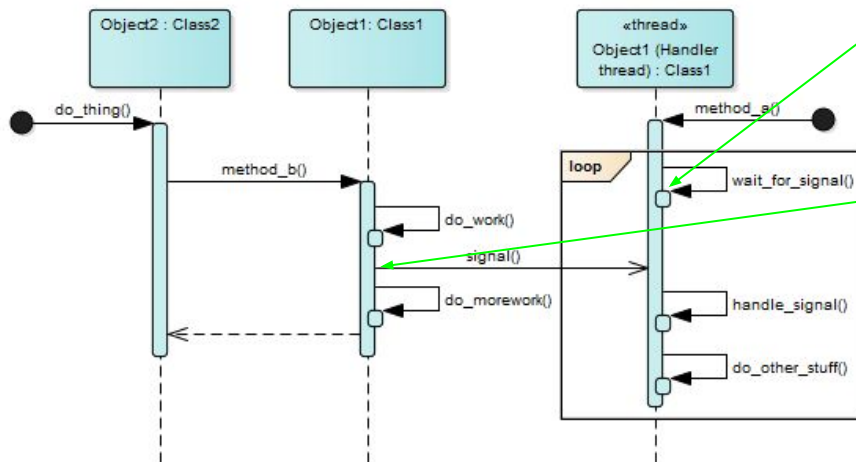
Primitive	Primary Purpose	Scope (Thread/Process)	Performance	Key Characteristics & Use Cases
lock / Monitor	Mutual Exclusion	Thread	Very High	• Most common. • Ensures only one thread enters a critical section. • Purely for intra-process synchronization.
Mutex	Mutual Exclusion	Process	Low	• Can be named for system-wide synchronization. • Heavier than lock. • Use for inter-process communication (IPC).
Semaphore	Resource Throttling	Process	Medium	• Allows a specified number of threads to access a resource pool. • Can be named for inter-process use. • Ideal for limiting concurrency (e.g., database connections).
SemaphoreSlim	Resource Throttling	Thread	High	• Lightweight version of Semaphore. • Recommended for intra-process throttling. • Supports async/await with WaitAsync().
ReaderWriterLockSlim	Optimized Read/Write	Thread	High (for reads)	• Allows multiple readers OR a single writer. • Boosts performance when reads vastly outnumber writes. • Prefer over the older ReaderWriterLock.
AutoResetEvent	Signaling	Thread	Medium	• A gate that lets one waiting thread through per signal. • Resets automatically after releasing a thread.
ManualResetEvent	Signaling	Thread	Medium	• A gate that, once signaled, lets all waiting threads through. • Stays open until manually reset.
ManualResetEventSlim	Signaling	Thread	High	• Lightweight version for intra-process signaling. • Spins briefly before waiting, for better performance.
Barrier	Phased Operations	Thread	Medium	• Allows multiple threads to run in phases. • All threads must signal arrival at the barrier before any can proceed. • Ideal for parallel algorithms.
CountdownEvent	Dynamic Signaling	Thread	Medium	• A signal is set when a counter reaches zero. • Useful for fork/join scenarios where the number of work items is dynamic.



Ключевые особенности внутреннего устройства



WaitHandle



```

1 using System;
2 using System.Threading;
3
4 class Class1
5 {
6     private readonly AutoResetEvent signal = new(false);
7
8     public Class1()
9     {
10         new Thread(() =>
11         {
12             while (true)
13             {
14                 Console.WriteLine("[Handler] Waiting...");
15                 signal.WaitOne();
16                 Console.WriteLine("[Handler] Handling signal...");
17                 Console.WriteLine("[Handler] Doing other stuff...");
18             }
19         })
20         { IsBackground = true }.Start();
21     }
22
23     public void MethodB()
24     {
25         Console.WriteLine("[Main] Doing work...");
26         Console.WriteLine("[Main] Signaling handler...");
27         signal.Set();
28         Console.WriteLine("[Main] Doing more work...");
29     }
30 }
31
32 class Class2
33 {
34     private readonly Class1 obj1;
35     public Class2(Class1 o) => obj1 = o;
36     public void DoThing()
37     {
38         Console.WriteLine("[Class2] do_thing()");
39         obj1.MethodB();
40     }
41 }
42
43 class Program
44 {
45     static void Main()
46     {
47         var obj1 = new Class1();
48         var obj2 = new Class2(obj1);
49         obj2.DoThing();
50
51         Console.WriteLine("Press Enter to exit...");
52         Console.ReadLine();
53     }
54 }

```

В чем сила Slim версий примитивов

Работа в **User-mode** режиме, позволяющая **оставаться потоку в состоянии Running** некоторое количество времени, **ожидая сигнала Wait** для доступа в критическую секцию. Эта возможность и продолжительность определяются типом **SpinWait**



<https://learn.microsoft.com/en-us/dotnet/api/system.threading.spinwait?view=net-9.0>

Best practice при использовании slim-версий

Не используй в критических секциях какие-либо типы обращающиеся к API OS, например :

- **Console.WriteLine**("Something")
- **Thread.Sleep**(TimeInMilliseconds) - даже 0 приведет к Switch Context

Для эмуляции работы вместо **Thread.Sleep()** используй работу в User mode, например :

- **Thread.SpinWait**(int iterations)
- **SpinWait** тип , позволяющий задать количество холостых тиков процессора , в том числе и по условию

Если в критической секции идет работа с типами, обращающимися к API OS, и вам не нужны асинхронные способы блокировки, или вы обеспечиваете межпроцессное взаимодействие - используйте классические версии примитивов, без Slim

SemaphoreSlim

SemaphoreSlim

♦ Semaphore vs SemaphoreSlim		
Feature / Aspect	Semaphore	SemaphoreSlim
Introduced	.NET 1.1	.NET 4.0
Scope	Kernel object (can be named, usable across processes)	In-process only (not visible across processes)
Performance	Heavier (kernel transitions)	Faster (mostly user-mode, lightweight)
Async support	✗ No (<code>WaitOne</code> only, blocking)	✓ Yes (<code>WaitAsync</code> , works with <code>async/await</code>)
Cross-process synchronization	✓ Yes (via named semaphore)	✗ No
Constructor parameters	<code>Semaphore(initialCount, maxCount, string? name)</code>	<code>SemaphoreSlim(initialCount, maxCount)</code>
When to use	Inter-process synchronization, legacy interop	Modern in-process synchronization, async code
Typical overhead	~10-50x slower than <code>SemaphoreSlim</code> in low-contention scenarios	Very low, designed for high-performance concurrency
Wait method	<code>WaitOne()</code> (blocking)	<code>Wait()</code> and <code>WaitAsync()</code>
Release method	<code>Release()</code>	<code>Release()</code>
Thread safety	Yes	Yes
Cancellation support	✗ No	✓ Yes (<code>WaitAsync(CancellationToken)</code>)

SemaphoreSlim

Category	Member	Description	Why It's Useful for Single Process
Properties	<code>CurrentCount</code>	Gets the number of remaining entries that can be entered before the semaphore blocks.	Useful for monitoring and debugging concurrency limits in a lightweight way.
	<code>AvailableWaitHandle</code>	Returns a <code>WaitHandle</code> that can be used to wait on the semaphore (mainly for interoperability).	Provides optional compatibility with APIs expecting a <code>WaitHandle</code> , without heavy kernel object creation.
Constructors	<code>SemaphoreSlim(int initialCount)</code>	Initializes with a specified number of requests that can be granted concurrently.	Lightweight and easy setup for local synchronization limits.
	<code>SemaphoreSlim(int initialCount, int maxCount)</code>	Initializes with an initial and maximum number of entries.	Allows flexible resource-management boundaries within a single app domain.
Core Methods (Synchronous)	<code>Wait()</code>	Blocks the current thread until it can enter the semaphore.	Efficient blocking for intra-process synchronization without kernel transitions.
	<code>Wait(int millisecondsTimeout)</code>	Blocks for a specified time before timing out.	Avoids indefinite waits — useful in responsive apps.
	<code>Wait(CancellationTokens)</code>	Allows cancellation during the wait.	Integrates cleanly with task cancellation patterns in .NET.
Core Methods (Asynchronous)	<code>WaitAsync()</code>	Asynchronously waits to enter the semaphore.	Non-blocking , ideal for async code (e.g., async I/O or web servers).
	<code>WaitAsync(TimeSpan timeout) / WaitAsync(int millisecondsTimeout)</code>	Asynchronously waits with timeout.	Provides fine-grained async control with timeout semantics.
	<code>WaitAsync(CancellationTokens)</code>	Asynchronously waits and can be cancelled.	Integrates naturally into modern async/await workflows.
Release Methods	<code>Release()</code>	Releases one count to the semaphore, allowing another thread/task to enter.	Fast in-memory operation (no OS semaphore handle).
	<code>Release(int releaseCount)</code>	Releases multiple entries at once.	Useful for batch resource releases.
Other Methods	<code>Dispose()</code>	Frees resources used by the semaphore.	Lightweight cleanup (no unmanaged kernel handles).

<https://learn.microsoft.com/en-us/dotnet/standard/threading/semaphore-and-semaphoreslim>

SemaphoreSlim пример использования

```
1  class BoundedBuffer<T>
2  {
3      private readonly Queue<T> queue = new Queue<T>();
4      private readonly int capacity;
5      private readonly SemaphoreSlim emptySlots;
6      private readonly SemaphoreSlim filledSlots = new SemaphoreSlim(0, int.MaxValue);
7      private readonly SemaphoreSlim lockSem = new SemaphoreSlim(1);
8
9      public BoundedBuffer(int capacity)
10     {
11         this.capacity = capacity;
12         emptySlots = new SemaphoreSlim(capacity, capacity);
13     }
14
15     public async Task ProduceAsync(T item)
16     {
17         // wait until there's an empty slot
18         await emptySlots.WaitAsync();
19
20         await lockSem.WaitAsync();
21         queue.Enqueue(item);
22         Console.WriteLine($"Produced: {item}");
23         lockSem.Release();
24
25         // signal that a new item is available
26         filledSlots.Release();
27     }
28
29     public async Task<T> ConsumeAsync(Cancellation_token cancellationToken)
30     {
31         // wait until there's something to consume
32         await filledSlots.WaitAsync(cancellationToken);
33
34         T item;
35
36         await lockSem.WaitAsync();
37         item = queue.Dequeue();
38         lockSem.Release();
39
40         // signal that a slot is free
41         emptySlots.Release();
42
43         return item;
44     }
45 }
```



SemaphoreSlim benchmark

```
1 private Semaphore _semaphore;  
2 private SemaphoreSlim _semaphoreSlim;  
3 private const int Iterations = 1000;  
4  
5 [GlobalSetup]  
6 public void Setup()  
7 {  
8     _semaphore = new Semaphore(10, 10);  
9     _semaphoreSlim = new SemaphoreSlim(10, 10);  
10 }  
11  
12 [Benchmark(Baseline = true)]  
13 public void Semaphore_Synchronous()  
14 {  
15     for (int i = 0; i < Iterations; i++)  
16     {  
17         _semaphore.WaitOne();  
18         // Simulate work  
19         Thread.SpinWait(100);  
20         _semaphore.Release();  
21     }  
22 }  
23  
24 [Benchmark]  
25 public void SemaphoreSlim_Synchronous()  
26 {  
27     for (int i = 0; i < Iterations; i++)  
28     {  
29         _semaphoreSlim.Wait();  
30         // Simulate work  
31         Thread.SpinWait(100);  
32         _semaphoreSlim.Release();  
33     }  
34 }
```

SemaphoreSlim benchmark

BenchmarkDotNet v0.15.4, Windows 11 (10.0.26100.6584/24H2/2024Update/HudsonValley)
AMD Ryzen 7 5800X 3.80GHz, 1 CPU, 16 logical and 8 physical cores
.NET SDK 9.0.203
[Host] : .NET 9.0.4 (9.0.4, 9.0.425.16305), X64 RyuJIT x86-64-v3
Job-NTRUNJ : .NET 9.0.4 (9.0.4, 9.0.425.16305), X64 RyuJIT x86-64-v3

IterationCount=5 WarmupCount=3

Method	Mean	Error	StdDev	Ratio
Semaphore_Synchronous	4,812.4  μs	57.37 μs	8.88 μs	1.00
SemaphoreSlim_Synchronous	2,777.8 μs	94.55 μs	14.63 μs	0.58
SemaphoreSlim_Asynchronous	4,128.6 μs	35.11 μs	9.12 μs	0.86
Semaphore_HighContention	394.4 μs	69.93 μs	18.16 μs	0.08
SemaphoreSlim_HighContention	242.2 μs	7.05 μs	1.83 μs	0.05
SemaphoreSlim_HighContentionAsync	253.1 μs	3.64 μs	0.95 μs	0.05

LIVE if necessary)

ReaderWriterLockSlim

ReaderWriterLockSlim



Summary

Feature	ReaderWriterLock	ReaderWriterLockSlim
Introduced	.NET 1.1	.NET 3.5
Performance	Slow (kernel-based)	Fast (user-mode mostly)
Async support	✗	✗ (but can be wrapped in async-friendly code)
Starvation risk	Higher	Lower
Upgradeable lock	Yes	Yes
Recommended	✗ Legacy only	✓ Use this

ReaderWriterLockSlim

Category	Member	Description	Why It's Useful / Improvement Over <code>ReaderWriterLock</code> 
Properties	<code>CurrentReadCount</code>	Gets the total number of threads that have entered the lock in read mode.	Useful for diagnostics and debugging concurrency patterns.
	<code>IsReadLockHeld</code>	Indicates if the current thread has entered the lock in read mode.	Allows easy validation of read-lock ownership in complex operations.
	<code>IsWriteLockHeld</code>	Indicates if the current thread has entered the lock in write mode.	Helps prevent accidental re-entrance or unsafe write operations.
	<code>IsUpgradeableReadLockHeld</code>	True if the current thread has an upgradeable read lock.	Enables controlled promotion from read to write access.
	<code>RecursiveReadCount</code>	Gets how many times the current thread entered the lock in read mode.	Supports recursive locking (configurable via constructor).
	<code>RecursiveWriteCount</code>	Gets how many times the current thread entered the lock in write mode.	Enables re-entrant write locks when allowed.
	<code>WaitingReadCount</code>	Number of threads waiting to enter in read mode.	Useful for performance monitoring.
	<code>WaitingWriteCount</code>	Number of threads waiting to enter in write mode.	Helps detect contention hotspots.
	<code>WaitingUpgradeCount</code>	Number of threads waiting to enter in upgradeable read mode.	Useful for tuning concurrency behavior.

<https://learn.microsoft.com/en-us/dotnet/api/system.threading.readerwriterlockslim?view=net-8.0>



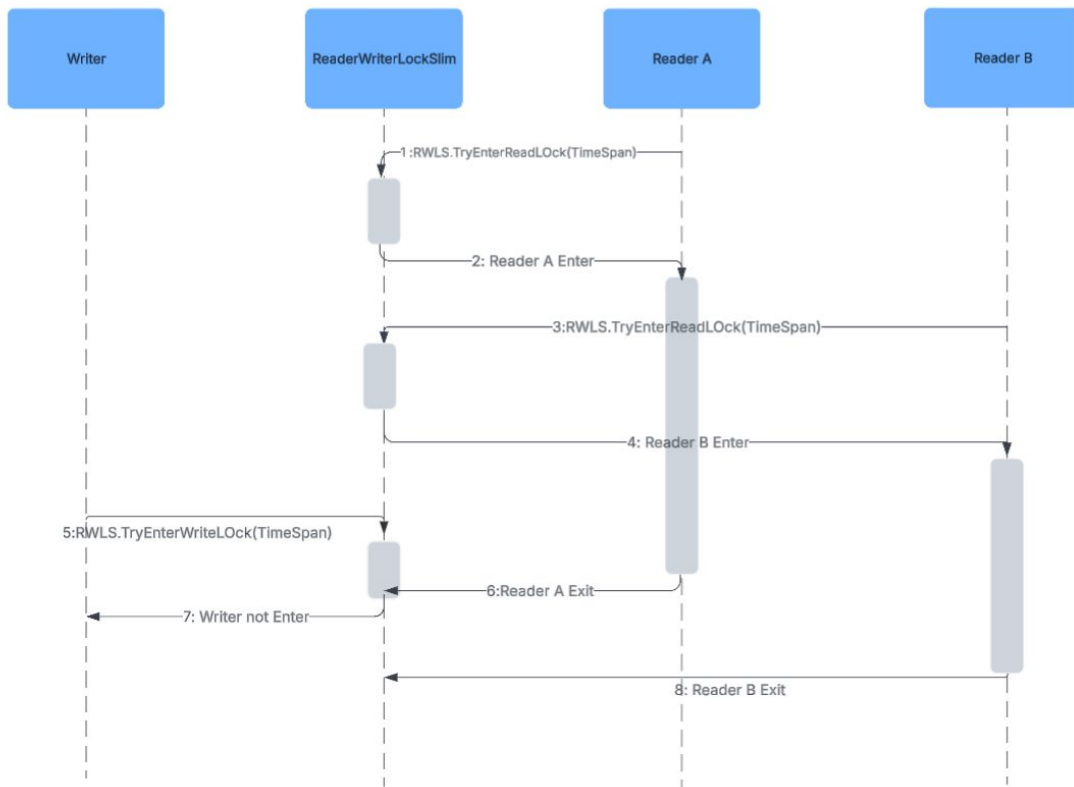
ReaderWriterLockSlim

Constructors	<code>ReaderWriterLockSlim()</code>	Initializes a new instance with default recursion policy.	Lightweight and efficient initialization.
	<code>ReaderWriterLockSlim(LockRecursionPolicy recursionPolicy)</code>	Enables recursive or non-recursive locking behavior.	Provides more control than the older <code>ReaderWriterLock</code> .
Core Methods (Read)	<code>EnterReadLock()</code>	Blocks until the thread can enter read mode.	Fast in-memory read locking with minimal contention.
	<code>TryEnterReadLock(int millisecondsTimeout)</code>	Tries to enter read mode within a given timeout.	Prevents deadlocks by avoiding indefinite waits.
	<code>ExitReadLock()</code>	Releases the read lock.	Low-overhead exit — no kernel transitions.
Core Methods (Write)	<code>EnterWriteLock()</code>	Blocks until the thread can enter write mode.	Exclusive access for write operations.
	<code>TryEnterWriteLock(int millisecondsTimeout)</code>	Tries to enter write mode with timeout.	Increases responsiveness in high-contention environments.
	<code>ExitWriteLock()</code>	Releases the write lock.	Lightweight compared to kernel-based locks.
Upgradeable Lock Methods	<code>EnterUpgradeableReadLock()</code>	Allows a thread to read and later upgrade to write.	Avoids deadlocks when upgrading.
	<code>TryEnterUpgradeableReadLock(int millisecondsTimeout)</code>	Attempts to enter upgradeable mode with timeout.	Enables safe and flexible promotion from read to write.
	<code>ExitUpgradeableReadLock()</code>	Releases the upgradeable read lock.	Keeps lock management clear and predictable.
Other Methods	<code>Dispose()</code>	Releases all resources used by the lock.	Lightweight managed cleanup.

<https://learn.microsoft.com/en-us/dotnet/api/system.threading.readerwriterlockslim?view=net-8.0>



ReaderWriterLockSlim - проблема голодания



ReaderWriterLockSlim - производительность

```
1 [Benchmark(Description = "RWLock - High Contention")]
2 public void ReaderWriterLock_HighContention()
3 {
4     var tasks = new Task[ThreadCount];
5     int opsPerThread = 500;
6
7     for (int i = 0; i < ThreadCount; i++)
8     {
9         tasks[i] = Task.Run(() =>
10         {
11             for (int j = 0; j < opsPerThread; j++)
12             {
13                 // Multiple rapid lock acquisitions
14                 _rwLock.AcquireReaderLock(TimeoutVal);
15                 try
16                 {
17                     int temp = _data;
18                 }
19                 finally
20                 {
21                     _rwLock.ReleaseReaderLock();
22                 }
23
24                 if (j % 10 == 0)
25                 {
26                     _rwLock.AcquireWriterLock(TimeoutVal);
27                     try
28                     {
29                         _data++;
30                     }
31                     finally
32                     {
33                         _rwLock.ReleaseWriterLock();
34                     }
35                 }
36             }
37         });
38     }
39
40     Task.WaitAll(tasks);
41 }
```

```
43 [Benchmark(Description = "RWLockSlim - High Contention")]
44 public void ReaderWriterLockSlim_HighContention()
45 {
46     var tasks = new Task[ThreadCount];
47     int opsPerThread = 500;
48
49     for (int i = 0; i < ThreadCount; i++)
50     {
51         tasks[i] = Task.Run(() =>
52         {
53             for (int j = 0; j < opsPerThread; j++)
54             {
55                 // Multiple rapid lock acquisitions
56                 if(_rwLockSlim.TryEnterReadLock(TimeoutVal))
57                 {try
58                     {
59                         int temp = _data;
60                     }
61                     finally
62                     {
63                         _rwLockSlim.ExitReadLock();
64                     }
65                 }
66
67                 if (j % 10 == 0)
68                 {
69                     if(_rwLockSlim.TryEnterWriteLock(TimeoutVal))
70                     {try
71                         {
72                             _data++;
73                         }
74                         finally
75                         {
76                             _rwLockSlim.ExitWriteLock();
77                         }
78                     }
79                 }
80             }
81         });
82     }
83
84     Task.WaitAll(tasks);
85 }
```

ReaderWriterLockSlim - производительность

BenchmarkDotNet v0.15.4, Windows 11 (10.0.26100.6584/24H2/2024Update/HudsonValley)

AMD Ryzen 7 5800X 3.80GHz, 1 CPU, 16 logical and 8 physical cores

.NET SDK 9.0.203

[Host] : .NET 9.0.4 (9.0.4, 9.0.425.16305), X64 RyuJIT x86-64-v3

Job-NTRUNJ : .NET 9.0.4 (9.0.4, 9.0.425.16305), X64 RyuJIT x86-64-v3

IterationCount=5 WarmupCount=3

Method	ThreadCount	TimeOutVal	ReadPercentage	Mean	Error	StdDev	Ratio	RatioSD	Allocated	Alloc Ratio
'RWLock - Concurrent'	2	50	90	647.10 µs	9.637 µs	2.503 µs	0.91	0.01	3021 B	2.97
'RWLockSlim - Concurrent'	2	50	90	709.79 µs	24.573 µs	3.803 µs	1.00	0.01	1016 B	1.00
'RWLock - High Contention'	2	50	90	48.31 µs	2.637 µs	0.685 µs	0.07	0.00	408 B	0.40
'RWLockSlim - High Contention'	2	50	90	32.03 µs	1.030 µs	0.159 µs	0.05	0.00	408 B	0.40
'RWLockSlim - Upgradable (Concurrent)'	2	50	90	44.71 µs	1.130 µs	0.294 µs	0.06	0.00	408 B	0.40
'RWLock - Multiple Readers'	2	50	90	3,056.95 µs	113.038 µs	29.356 µs	4.31	0.04	408 B	0.40
'RWLockSlim - Multiple Readers'	2	50	90	3,048.93 µs	131.786 µs	34.225 µs	4.30	0.05	408 B	0.40
'RWLock - Concurrent'	2	50	95	723.34 µs	699.920 µs	181.767 µs	1.19	0.27	3656 B	3.60
'RWLockSlim - Concurrent'	2	50	95	607.07 µs	28.957 µs	7.520 µs	1.00	0.02	1016 B	1.00
'RWLock - High Contention'	2	50	95	49.53 µs	1.213 µs	0.315 µs	0.08	0.00	408 B	0.40
'RWLockSlim - High Contention'	2	50	95	34.14 µs	1.718 µs	0.446 µs	0.06	0.00	408 B	0.40
'RWLockSlim - Upgradable (Concurrent)'	2	50	95	38.48 µs	0.908 µs	0.236 µs	0.06	0.00	408 B	0.40
'RWLock - Multiple Readers'	2	50	95	3,009.36 µs	70.847 µs	10.964 µs	4.96	0.06	408 B	0.40
'RWLockSlim - Multiple Readers'	2	50	95	4,606.38 µs	265.609 µs	68.978 µs	7.59	0.13	408 B	0.40
'RWLock - Concurrent'	2	50	99	427.76 µs	18.973 µs	2.936 µs	0.92	0.01	2410 B	2.37
'RWLockSlim - Concurrent'	2	50	99	466.25 µs	25.789 µs	6.697 µs	1.00	0.02	1016 B	1.00
'RWLock - High Contention'	2	50	99	53.90 µs	2.281 µs	0.592 µs	0.12	0.00	408 B	0.40
'RWLockSlim - High Contention'	2	50	99	38.44 µs	0.690 µs	0.179 µs	0.08	0.00	408 B	0.40
'RWLockSlim - Upgradable (Concurrent)'	2	50	99	38.87 µs	0.547 µs	0.142 µs	0.08	0.00	408 B	0.40
'RWLock - Multiple Readers'	2	50	99	3,041.59 µs	77.742 µs	20.189 µs	6.52	0.09	408 B	0.40
'RWLockSlim - Multiple Readers'	2	50	99	4,558.95 µs	336.049 µs	87.271 µs	9.78	0.21	408 B	0.40



ReaderWriterLockSlim - производительность

BenchmarkDotNet v0.15.4, Windows 11 (10.0.26100.6584/24H2/2024Update/HudsonValley)

AMD Ryzen 7 5800X 3.80GHz, 1 CPU, 16 logical and 8 physical cores

.NET SDK 9.0.203

[Host] : .NET 9.0.4 (9.0.4, 9.0.425.16305), X64 RyuJIT x86-64-v3

Job-NTRUNJ : .NET 9.0.4 (9.0.4, 9.0.425.16305), X64 RyuJIT x86-64-v3

IterationCount=5 WarmupCount=3

Method	ThreadCount	TimeOutVal	ReadPercentage	Mean	Error	StdDev	Ratio	RatioSD	Allocated	Alloc Ratio
'RWriteLock - Concurrent'	16	50	90	13,189.33 µs	1,406.261 µs	217.620 µs	2.43	0.04	6988 B	1.09
'RWriteLockSlim - Concurrent'	16	50	90	5,420.41 µs	78.661 µs	20.428 µs	1.00	0.00	6392 B	1.00
'RWriteLock - High Contention'	16	50	90	5,757.66 µs	1,030.785 µs	159.515 µs	1.06	0.03	1845 B	0.29
'RWriteLockSlim - High Contention'	16	50	90	458.02 µs	5.586 µs	1.451 µs	0.08	0.00	1528 B	0.24
'RWriteLockSlim - Upgradeable (Concurrent)'	16	50	90	936.67 µs	10.542 µs	2.738 µs	0.17	0.00	1528 B	0.24
'RWriteLock - Multiple Readers'	16	50	90	3,388.08 µs	277.030 µs	71.944 µs	0.63	0.01	1528 B	0.24
'RWriteLockSlim - Multiple Readers'	16	50	90	3,347.09 µs	112.053 µs	17.340 µs	0.62	0.00	1528 B	0.24
'RReadLock - Concurrent'	16	50	95	8,632.98 µs	987.638 µs	256.487 µs	2.00	0.06	7325 B	1.15
'RReadLockSlim - Concurrent'	16	50	95	4,307.19 µs	81.531 µs	21.173 µs	1.00	0.01	6392 B	1.00
'RReadLock - High Contention'	16	50	95	6,045.43 µs	570.423 µs	148.137 µs	1.40	0.03	1980 B	0.31
'RReadLockSlim - High Contention'	16	50	95	400.20 µs	4.303 µs	1.117 µs	0.09	0.00	1528 B	0.24
'RReadLockSlim - Upgradeable (Concurrent)'	16	50	95	1,016.58 µs	19.486 µs	5.060 µs	0.24	0.00	1528 B	0.24
'RReadLock - Multiple Readers'	16	50	95	3,385.14 µs	185.961 µs	48.294 µs	0.79	0.01	1528 B	0.24
'RReadLockSlim - Multiple Readers'	16	50	95	3,484.15 µs	636.045 µs	98.429 µs	0.81	0.02	1528 B	0.24
'RReadLock - Concurrent'	16	50	99	3,179.84 µs	205.769 µs	53.438 µs	1.11	0.02	7575 B	1.19
'RReadLockSlim - Concurrent'	16	50	99	2,875.56 µs	89.550 µs	23.256 µs	1.00	0.01	6392 B	1.00
'RReadLock - High Contention'	16	50	99	5,837.52 µs	882.232 µs	229.113 µs	2.03	0.07	1840 B	0.29
'RReadLockSlim - High Contention'	16	50	99	436.56 µs	6.585 µs	1.710 µs	0.15	0.00	1528 B	0.24
'RReadLockSlim - Upgradeable (Concurrent)'	16	50	99	1,046.83 µs	23.331 µs	6.059 µs	0.36	0.00	1528 B	0.24
'RReadLock - Multiple Readers'	16	50	99	3,465.70 µs	483.995 µs	125.692 µs	1.21	0.04	1528 B	0.24
'RReadLockSlim - Multiple Readers'	16	50	99	3,563.61 µs	818.254 µs	212.498 µs	1.24	0.07	1528 B	0.24



LIVE

ManualResetEventSlim

ManualResetEventSlim

ManualResetEvent vs ManualResetEventSlim

Aspect	ManualResetEvent	ManualResetEventSlim
Implementation	Uses a kernel object (Win32 event)	Hybrid: spins briefly in user mode, then falls back to kernel wait
Performance	Slower (context switches into kernel space on wait)	Faster for short, frequent waits because it avoids kernel transitions initially
CPU usage	Lower under contention (threads sleep in kernel)	Higher for very long waits (due to spinning) but much faster for short waits
Cross-process support	✅ Yes (can create a named event visible system-wide)	❌ No (in-process only)
Async/await compatibility	❌ Blocking only (<code>WaitOne</code>)	❌ Blocking only (<code>Wait</code>), but lighter
Memory footprint	Heavier	Lighter
When signaled (<code>Set</code>)	Releases all waiters, stays signaled until <code>Reset</code>	Same behavior
Cancellation support	❌ No built-in	✅ Has <code>Wait(CancellationTokens)</code> overloads
Use cases	- Inter-process synchronization - Legacy Win32 interop - Low CPU but higher latency acceptable	- High-performance in-process coordination - Scenarios with many short waits (e.g., parallel algorithms) - When cancellation tokens are useful

ManualResetEventSlim

ManualResetEventSlim — Key Properties & Methods			
Category	Member / Feature	Description	Why It's Preferable in Single Process
Basic Behavior	<code>Set()</code>	Signals the event, releasing all waiting threads.	Fast managed signaling without kernel transition.
	<code>Reset()</code>	Returns the event to a nonsignaled state.	Allows reuse; operates entirely in user mode.
	<code>Wait()</code>	Blocks the current thread until signaled.	Performs spin-wait first, avoiding context switches if signal comes quickly.
	<code>Wait(TimeSpan)</code> / <code>Wait(int)</code>	Waits with timeout.	Fine control over responsiveness; still efficient in-process.
	<code>Wait(CancellationToken)</code>	Waits but can be canceled via token.	Integrates cleanly with modern .NET cancellation patterns.
State & Info	<code>IsSet</code> (property)	Indicates whether the event is currently signaled.	Allows quick checks without entering wait state.
	<code>WaitHandle</code> (property)	Lazily creates an underlying <code>ManualResetEvent</code> .	Kernel handle created only if absolutely needed (e.g., for legacy APIs).
	<code>SpinCount</code> (property)	Number of spin iterations before kernel wait. Default: environment-optimized.	Enables CPU-bound spin-waiting for micro-latency synchronization.
Threading & Performance	Spin-Wait Phase	Performs short busy-wait before blocking.	Reduces context switching overhead for very short waits.
	Kernel-Wait Phase	Falls back to kernel blocking if signal takes longer.	Provides balance between CPU use and latency.
Disposal	<code>Dispose()</code>	Releases internal resources and handle if created.	Lightweight — minimal cleanup needed unless <code>WaitHandle</code> was created.
Constructors	<code>ManualResetEventSlim()</code>	Initializes as nonsignaled.	Fully managed — no kernel handle at creation.
	<code>ManualResetEventSlim(bool initialState)</code>	Allows starting in signaled or nonsignaled state.	Immediate use with minimal allocation.

<https://learn.microsoft.com/ru-ru/dotnet/api/system.threading.manualreseteventSlim?view=net-8.0>



LIVE

Список материалов для изучения

<https://gitlab.com/otus-demo/in-depth/sync-primitives>

<https://learn.microsoft.com/en-us/dotnet/standard/threading/semaphore-and-semaphoreslim>

<https://learn.microsoft.com/en-us/dotnet/api/system.threading.readerwriterlock?view=net-9.0>

<https://learn.microsoft.com/en-us/dotnet/api/system.threading.readerwriterlockslim?view=net-8.0>

<https://learn.microsoft.com/ru-ru/dotnet/api/system.threading.manualreseteventslim?view=net-8.0>

<https://medium.com/c-sharp-programming/exploring-the-new-lock-object-in-c-3410071860bc>

<https://www.baeldung.com/cs/readers-writers-problem>

<https://github.com/microsoft/referencesource/blob/main/mscorlib/system/threading/SemaphoreSlim.cs>

<https://github.com/microsoft/referencesource/blob/main/mscorlib/system/threading/SpinWait.cs>

<https://benchmarkdotnet.org/articles/guides/getting-started.html>

<https://benchmarkdotnet.org/articles/configs/jobs.html>

**Подведём
итоги занятия -
рефлексия**

Вопросы



если есть вопросы



если вопросов нет

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

**You are our
OTUS heroes**

